

RTL With Verilog LAB Week #4

Introduction to VGA Controller Interfaced with SRAM

SRAM

Static Random Access Memory (SRAM) is a type of volatile memory that stores data using flip-flops. Unlike DRAM, SRAM does not require periodic refreshing to retain stored data as long as power is supplied. It provides high-speed data access, low latency, and reliable performance, making it suitable for applications such as cache memory, frame buffers, and display systems. In this project, SRAM is used to store the pixel data that is read by the VGA Controller for display on the monitor.

VGA Controller

A VGA (Video Graphics Array) Controller is a digital circuit responsible for generating the timing signals required to drive a VGA monitor. It produces the Horizontal Synchronization (HSYNC) and Vertical Synchronization (VSYNC) signals while simultaneously outputting RGB color data to create the display image. During the active display region, the controller reads pixel data from memory and sends it to the monitor. During the blanking intervals, the RGB outputs remain inactive while synchronization signals are generated.

The VGA Controller implemented in this project is designed for an **800 × 600 resolution** operating at a **60 Hz refresh rate** using a **50 MHz clock**. It provides **12-bit RGB color output**, with 4 bits allocated to each of the Red, Green, and Blue channels. The controller interfaces with SRAM through a read-only memory interface and automatically generates sequential memory addresses to fetch pixel data. All internal registers and counters are initialized to known values, ensuring stable simulation and reliable hardware operation.

Design Code:

```
module vga_controller (  
  
    input wire clk,  
  
    input wire rst_n,  
  
    input wire [11:0] data_in,  
  
    output wire hsync,  
  
    output wire vsync,  
  
    output wire [11:0] rgb_out,
```

```
output wire [18:0] sram_addr,  
  
output wire sram_ce,  
  
output wire sram_we,  
  
output wire sram_oe  
);
```

```
parameter H_FRONT = 16;  
  
parameter H_SYNC = 80;  
  
parameter H_BACK = 88;  
  
parameter H_ACTIVE = 800;  
  
parameter H_TOTAL = 1040;
```

```
parameter V_FRONT = 1;  
  
parameter V_SYNC = 3;  
  
parameter V_BACK = 21;  
  
parameter V_ACTIVE = 600;  
  
parameter V_TOTAL = 625;
```

```
reg [10:0] h_count;  
  
reg [10:0] v_count;  
  
wire h_active, v_active;  
  
wire h_sync_signal, v_sync_signal;  
  
  
reg [18:0] addr_count;
```

```
reg [11:0] rgb_reg;
```

```
initial begin
```

```
    h_count = 11'b0;
```

```
    v_count = 11'b0;
```

```
    addr_count = 19'b0;
```

```
    rgb_reg = 12'b0;
```

```
end
```

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
```

```
        h_count <= 11'b0;
```

```
    end else begin
```

```
        if (h_count == H_TOTAL - 1)
```

```
            h_count <= 11'b0;
```

```
        else
```

```
            h_count <= h_count + 1'b1;
```

```
    end
```

```
end
```

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
```

```
        v_count <= 11'b0;
```

```
    end else begin
```

```
        if (h_count == H_TOTAL - 1) begin
```

```

        if (v_count == V_TOTAL - 1)

            v_count <= 11'b0;

        else

            v_count <= v_count + 1'b1;

        end

    end

end

always @(posedge clk or negedge rst_n) begin

    if (!rst_n) begin

        addr_count <= 19'b0;

    end else begin

        if (h_active && v_active) begin

            if (h_count >= H_BACK && h_count < H_BACK + H_ACTIVE)

                addr_count <= addr_count + 1'b1;

            end else if (h_count == H_TOTAL - 1 && v_count == V_TOTAL - 1) begin

                addr_count <= 19'b0;

            end

        end

    end

end

assign h_active = (h_count >= H_BACK && h_count < H_BACK + H_ACTIVE);

assign v_active = (v_count >= V_BACK && v_count < V_BACK + V_ACTIVE);

```

```
assign h_sync_signal = (h_count >= H_BACK + H_ACTIVE + H_FRONT &&  
    h_count < H_BACK + H_ACTIVE + H_FRONT + H_SYNC);
```

```
assign v_sync_signal = (v_count >= V_BACK + V_ACTIVE + V_FRONT &&  
    v_count < V_BACK + V_ACTIVE + V_FRONT + V_SYNC);
```

```
assign sram_ce = 1'b0;
```

```
assign sram_we = 1'b1;
```

```
assign sram_oe = h_active && v_active ? 1'b0 : 1'b1;
```

```
assign sram_addr = addr_count;
```

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
```

```
        rgb_reg <= 12'b0;
```

```
    end else if (h_active && v_active) begin
```

```
        rgb_reg <= data_in;
```

```
    end else begin
```

```
        rgb_reg <= 12'b0;
```

```
    end
```

```
end
```

```
assign hsync = h_sync_signal;
```

```
assign vsync = v_sync_signal;
```

```
assign rgb_out = rgb_reg;
```

```
endmodule
```

```
`timescale 1ns / 1ps
```

Test Bench :

```
module tb_vga_controller;
```

```
    parameter CLK_PERIOD = 20;
```

```
    reg clk;
```

```
    reg rst_n;
```

```
    reg [11:0] data_in;
```

```
    wire hsync;
```

```
    wire vsync;
```

```
    wire [11:0] rgb_out;
```

```
    wire [18:0] sram_addr;
```

```
    wire sram_ce;
```

```
    wire sram_we;
```

```
    wire sram_oe;
```

```
    reg [31:0] frame_count;
```

```
    reg [31:0] pixel_count;
```

```
initial begin
```

```
    clk = 1'b0;
```

```
    rst_n = 1'b0;
```

```
    data_in = 12'b0;
```

```
    frame_count = 32'b0;
```

```
    pixel_count = 32'b0;
```

```
    #10;
```

```
end
```

```
always #10 clk = ~clk;
```

```
initial begin
```

```
    #40;
```

```
    rst_n = 1'b1;
```

```
    #100;
```

```
    rst_n = 1'b0;
```

```
    #40;
```

```
    rst_n = 1'b1;
```

```
    #20;
```

```
end
```

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
```

```

    data_in <= 12'b0;

    pixel_count <= 32'b0;
end else begin

    if (sram_oe == 1'b0) begin

        case (sram_addr[10:8])

            3'b000: data_in <= 12'hF00;

            3'b001: data_in <= 12'h0F0;

            3'b010: data_in <= 12'h00F;

            3'b011: data_in <= 12'hFF0;

            3'b100: data_in <= 12'hF0F;

            3'b101: data_in <= 12'h0FF;

            3'b110: data_in <= 12'hFFF;

            3'b111: data_in <= 12'h000;

            default: data_in <= 12'h555;

        endcase

        pixel_count <= pixel_count + 1;

    end else begin

        data_in <= 12'b0;

    end

end

end

always @(posedge vsync or negedge rst_n) begin

    if (!rst_n) begin

        frame_count <= 32'b0;

```



```

    end else if (vsync) begin

        frame_count <= frame_count + 1;

    end

end

vga_controller DUT (

    .clk(clk),

    .rst_n(rst_n),

    .data_in(data_in),

    .hsync(hsync),

    .vsync(vsync),

    .rgb_out(rgb_out),

    .sram_addr(sram_addr),

    .sram_ce(sram_ce),

    .sram_we(sram_we),

    .sram_oe(sram_oe)

);

initial begin

    $display("Starting VGA Controller Testbench...");

    $display("Time\t\tHSync\tVSync\tRGB\t\tSRAM_Addr");

    $display("-----");

    #1000000;

```

```

$display("Simulation completed. Total frames: %d", frame_count);

$display("Total pixels displayed: %d", pixel_count);


$display("\nTest Summary:");
$display("-----");


if (frame_count > 0) begin

    $display("PASS: Frame counter incremented correctly");

end else begin

    $display("FAIL: Frame counter did not increment");

end


if (pixel_count > 0) begin

    $display("PASS: Pixel counter incremented correctly");

end else begin

    $display("FAIL: Pixel counter did not increment");

end


$finish;

end


always @(posedge clk) begin

    if (hsync || vsync) begin

        end

    end

end

```

```
initial begin
```

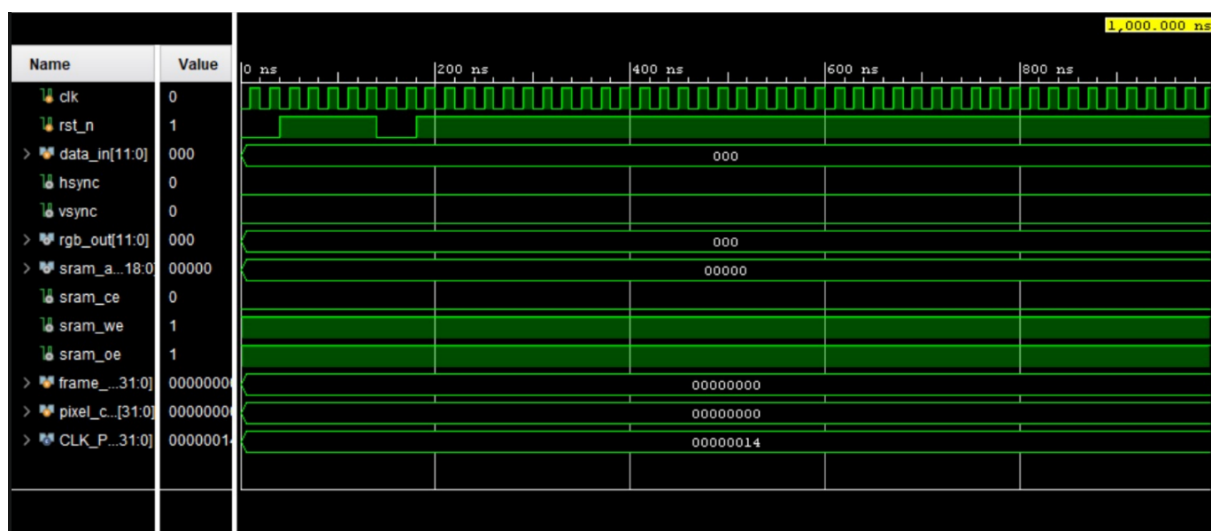
```
    $dumpfile("vga_controller.vcd");
```

```
    $dumpvars(0, tb_vga_controller);
```

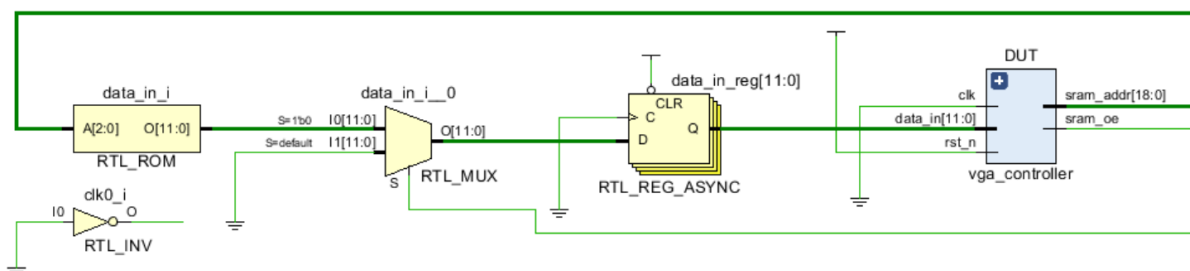
```
end
```

```
endmodule
```

Wave Form :



Synthesis:



Conclusion

This project successfully implements a VGA Controller interfaced with SRAM to display video output at a resolution. The controller correctly generates the required VGA timing signals, including **HSYNC**, **VSYNC**, and **RGB color outputs**, while reading pixel data from SRAM through an automatically generated address sequence.

Proper initialization of all internal registers and counters prevents undefined states, and the reset circuitry ensures consistent startup behavior. The modular architecture makes the design easy to understand, verify, and modify for different display resolutions or timing requirements. The testbench verifies the functionality of the complete system by generating pixel data, monitoring synchronization signals, and validating correct frame and pixel operation.

Overall, the design provides a reliable FPGA-based VGA display system that demonstrates the integration of SRAM with a VGA Controller. It serves as a strong foundation for future enhancements such as higher display resolutions, multiple frame buffers, image processing, and text or graphics overlay applications.